

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

**Асинхронная обработка, распределённое выполнение
задач,**

**планировщик и интеграция с внешней КИС
(платформа объявлений о недвижимости CIAN)**

по дисциплине **«Бизнес-логика программных систем»**

**Вариант №
3278**

Студенты: Кортых Константин Олегович
Соколов Артём Олегович

Группа: Р3318

Преподаватель: Кривоносов Егор Дмитриевич

Санкт-Петербург, 2026

1. Текст задания лабораторной работы

Доработать приложение из лабораторной работы №2, реализовав:

1. **асинхронное выполнение задач** с распределением бизнес-логики между несколькими вычислительными узлами;
2. **периодические операции** с использованием планировщика задач Spring (@Scheduled);
3. **интеграцию с внешней корпоративной информационной системой (EIS)** по технологии JCA (Jakarta Connectors).

Асинхронная обработка: модель «очередь сообщений»; провайдер — **RabbitMQ**; отправка и получение — **JMS API**.

Распределённая обработка: обработка сообщений на **двух независимых узлах** сервера приложений.

Планировщик: согласованные прецеденты реализованы через @Scheduled.

Интеграция EIS: взаимодействие с внешней КИС только через **JCA**; выбранная система согласуется с преподавателем.

Изменения бизнес-процесса отражены в модели, REST API и сценариях проверки. Демонстрация — на собственной инфраструктуре (Docker) или helios.

Содержание

1. Текст задания лабораторной работы	2
2. Согласованные прецеденты	4
2.1. Асинхронная обработка (очередь сообщений)	4
2.2. Периодические задачи (@Scheduled)	4
2.3. Интеграция с внешней КИС	4
3. Архитектура решения	5
3.1. Общая схема развёртывания (Docker)	5
3.2. Асинхронная обработка: JMS + RabbitMQ	5
3.3. Распределённая обработка на двух узлах	6
3.4. Планировщик задач Spring	7
3.5. Интеграция с EIS: JCA и Bitrix24	7
4. Изменения в модели данных и API	8
4.1. База данных	8
4.2. REST API	8
5. Структура исходного кода	8
6. Запуск стенда	8
7. Выводы	8

2. Согласованные прецеденты

2.1. Асинхронная обработка (очередь сообщений)

Прецедент	Обоснование	Реализация
Email-уведомление	Создание уведомления в БД не должно блокироваться отправкой письма; сбой SMTP не должен откатывать бизнес-транзакцию.	После commit в очередь JMS кладётся NotificationEvent; письмо отправляют worker-узлы.

Таблица 1. Прецедент асинхронной обработки.

Затрагиваются все сценарии, где вызывается `NotificationService.create` (публикация, продвижение, обращения, планировщики, смена статуса объявления и т.д.).

2.2. Периодические задачи (@Scheduled)

Прецедент (BPMN / требования)	Задача планировщика
Истечение срока публикации 30 дней	<code>ListingScheduler.sendArchivationSoonNotifications</code> — ежедневно 02:00, уведомление <code>ARCHIVATION_SOON</code> .
Архивация при неподтверждении актуальности	<code>ListingScheduler.autoArchiveExpiredListings</code> — ежедневно 03:00, перевод в <code>ARCHIVED</code> .
Еженедельная статистика по пользователю	<code>WeeklyStatsScheduler</code> — понедельник 09:00, отчёт в <code>cian_weekly_stats</code> + уведомление.

Таблица 2. Прецеденты планировщика Spring.

Планировщики активны только на узле **API** (`@Profile("!worker")`), не на worker-контейнерах.

2.3. Интеграция с внешней КИС

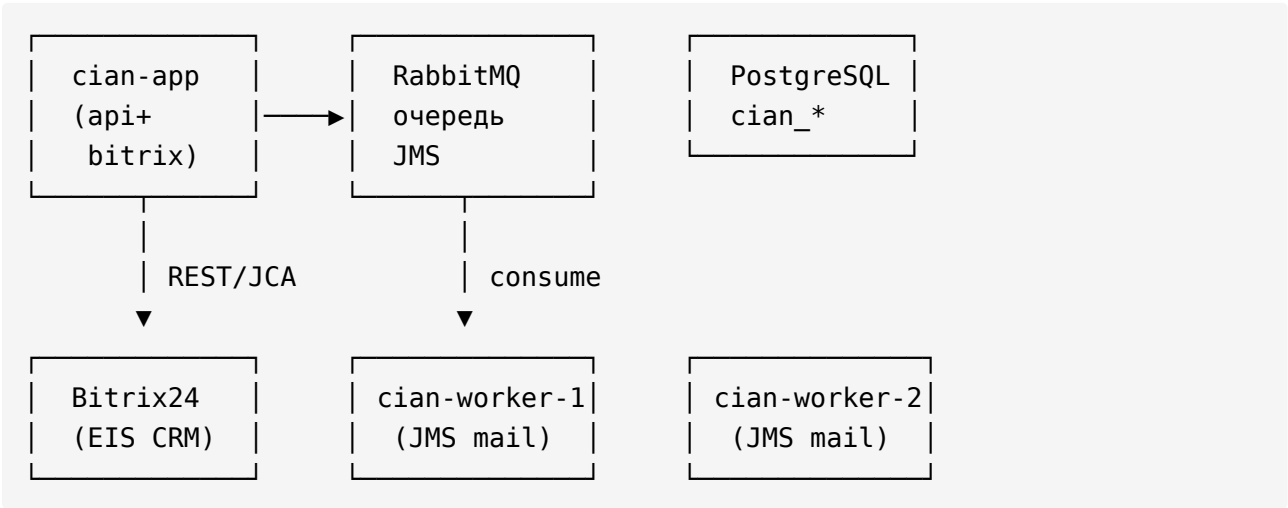
Внешняя система: облачная CRM **Bitrix24** (воронка «Cian», сделки, контакты, активности).

Направление	Прецедент CIAN	Сущность Bitrix
Outbound	Публикация объявления	Сделка + контакт продавца
Outbound	Заккрытие / архивация / админ-статус	Стадия сделки
Outbound	Успешная оплата TOP/PREMIUM	UF-поле продвижения
Outbound	Новое обращение (inquiry)	Activity на сделке
Inbound	Изменение стадии в CRM	Обновление Listing.status в PostgreSQL

Таблица 3. Согласованное отображение CIAN ↔ Bitrix24.

3. Архитектура решения

3.1. Общая схема развёртывания (Docker)



Конфигурация: `docker/docker-compose.yml`. Сборка: `docker/Dockerfile` (модули `cian + bitrix-jca-ra`).

3.2. Асинхронная обработка: JMS + RabbitMQ

Паттерн: Transactional Outbox на уровне приложения — запись уведомления в PostgreSQL в той же транзакции, что и бизнес-операция; **после успешного commit** событие публикуется в очередь.

Отправка (JMS):

- `NotificationService` → `NotificationEventPublisher` → `JmsTemplate.convertAndSend(...);`
- очередь: `app.messaging.notifications-queue` (по умолчанию `cian.notifications`);

- фабрика соединений: `RMQConnectionFactory (com.rabbitmq.jms)`, конфигурация в `JmsConfig`.

Получение (JMS):

- `NotificationEventListener` с `@JmsListener` на `worker`-узлах (`SPRING_PROFILES_ACTIVE=docker,worker`);
- обработчик вызывает `EmailNotificationService.send` (SMTP → Mailpit в dev).

Ключевые классы: `itmo.blps.messaging.*`, `itmo.blps.config.JmsConfig`, `itmo.blps.config.MessagingProperties`.

```
// NotificationService – публикация после commit
TransactionSynchronizationManager.registerSynchronization(
    new TransactionSynchronization() {
        @Override
        public void afterCommit() {
            notificationEventPublisher.publish(event);
        }
    });
```

Модель доставки: point-to-point (очередь JMS), конкурирующие потребители на двух `worker` — сообщение обрабатывает один из узлов.

3.3. Распределённая обработка на двух узлах

В `docker-compose.yml` определены:

1. `app` — REST API, JPA, планировщики, профиль `bitrix`, **без** `@JmsListener`;
2. `worker-1` и `worker-2` — один образ `cian-app:latest`, профиль `worker`: только JMS-listener и отправка почты.

Так достигается **физическое разделение** вычислительных узлов (отдельные контейнеры JVM).

Узел	Профили Spring	Нагрузка
<code>cian-app</code>	<code>docker, api, bitrix</code>	HTTP, cron, Bitrix JCA outbound/inbound
<code>cian-worker-1</code>	<code>docker, worker</code>	JMS → email
<code>cian-worker-2</code>	<code>docker, worker</code>	JMS → email

Таблица 4. Распределение ролей между контейнерами.

Распределённые транзакции (XA): в сценарии «уведомление + email» используется **локальная транзакция БД** и **асинхронная доставка после commit**, а не двухфазный commit между PostgreSQL и RabbitMQ. Это соответствует типовой практике «at-least-once» для почтовых уве-

домлений. Bitrix JCA-адаптер явно не поддерживает ХА (getXAResource → NotSupportedException).

3.4. Планировщик задач Spring

@EnableScheduling в CianApplication.

Класс	Cron	Действие
ListingScheduler	0 0 2 * * *	Уведомления об истечении срока размещения
ListingScheduler	0 0 3 * * *	Автоархивация после grace-period
WeeklyStatsScheduler	0 0 9 * * MON	Генерация отчётов за прошлую неделю

Таблица 5. Задачи @Scheduled.

3.5. Интеграция с EIS: JCA и Bitrix24

Интеграция вынесена в отдельный Gradle-модуль bitrix-jca-ra.

Компонент JCA	Класс / назначение
Resource Adapter	BitrixResourceAdapter — inbound polling crm.deal.list
Managed Connection Factory	BitrixManagedConnectionFactory — REST webhook URL
CCI	BitrixConnection / BitrixConnectionImpl — единственный путь к Bitrix REST
Connection Factory	BitrixConnectionFactory — для CrmSyncService
Message listener	BitrixEventListener → BitrixInboundSyncService
Embedded container	BitrixJcaEmbeddedContainer — запуск RA в Spring Boot

Таблица 6. Структура JCA-адаптера Bitrix24.

Прикладной слой: CrmSyncService, таблица связей cian_crm_links, хуки в ListingService, InquiryService, PaymentService. Синхронизация после commit (CrmSyncHelper). Защита от циклов — BitrixSyncContext.isInbound().

Backfill: при старте (BitrixStartupBackfill) в Bitrix выгружаются объявления из seed.sql (не DRAFT) и inquiries без связи.

Конфигурация: профиль bitrix, application-bitrix.yml, переменная BITRIX_REST_BASE_URL в docker/.env.

Приложение **не вызывает** Bitrix через RestTemplate напрямую — только через CCI BitrixConnection (требование JCA).

4. Изменения в модели данных и API

4.1. База данных

Добавлена таблица `cian_crm_links` (`entity_type`, `local_id`, `bitrix_id`, `last_sync_at`) — сопоставление сущностей CIAN и Bitrix. Скрипт: `src/sql/schema.sql`; `seed.sql` очищает и `cian_crm_links`.

4.2. REST API

Публичные контракты из ЛР №2 сохранены (`docs/API_ENDPOINTS.md`). Интеграция с Bitrix **не добавляет** обязательных публичных эндпоинтов для пользователя; обмен с EIS идёт через JCA (outbound/inbound polling).

5. Структура исходного кода

```
blps_lab2/
├─ bitrix-jca-ra/          # JCA Resource Adapter (ra.xml, RA, MCF, CCI)
├─ src/main/java/itmo/blps/
│   └─ config/            # JmsConfig, ListingScheduler,
WeeklyStatsScheduler, Bitrix*
│   └─ messaging/         # NotificationEvent, Publisher, Listener
│   └─ integration/crm/   # CrmSyncService, BitrixSyncContext
│   └─ service/           # бизнес-логика + хуки CRM
│   └─ ...
├─ docker/               # compose: app + 2 workers + rabbitmq + db +
mailpit
└─ docs/                 # API, БД, отчёт
```

6. Запуск стенда

```
# docker/.env
BITRIX_REST_BASE_URL=https://<portal>.bitrix24.ru/rest/1/<token>/

cd docker
docker compose up --build -d
```

Сервисы: приложение `http://localhost:8080`, RabbitMQ UI `http://localhost:15672` (`cian/cian`), Mailpit `http://localhost:8025`.

7. Выводы

В ходе работы платформа CIAN доработана до архитектуры с **разделением синхронного API и асинхронных worker-узлов** на базе **JMS/RabbitMQ**,

что позволяет масштабировать отправку email-уведомлений и изолировать сбои почтового канала от основных транзакций.

Планировщик Spring автоматизирует сценарии истечения срока публикации и еженедельной отчётности без участия пользователя, что соответствует диаграммам BPMN и требованиям R3.x.

Интеграция с Bitrix24 через JCA реализует двусторонний обмен: объявления и обращения отражаются в CRM как сделки и активности, изменения стадий в Bitrix возвращаются в статусы объявлений. Вся работа с внешней КИС сосредоточена в Resource Adapter и CCI, что выполняет учебное требование по Jakarta Connectors.